CO2 ASSESSMENT TOOL _3

NAME: M.SAI HARSHITHA
REG.NO: 192424408
COURSE CODE: DSA0304
COURSE NAME: NATURAL LANGUAGE PROCESSOR

QUESTION 1 — PORTER STEMMING ERRORS IN BIOMEDICAL TEXT

CODE:

```
import re
import pandas as pd
from nltk.stem import PorterStemmer
ps = PorterStemmer()
words = ["infection", "infectious", "infected", "infect"]
def analyze(word):
    stem = ps.stem(word)
    if word in ("infected",):
        morph_type = "Inflectional"
    elif word in ("infection", "infectious"):
        morph_type = "Derivational"
    else:
        morph_type = "Base form"
    return {
        "Original Word": word,
        "Porter Stem": stem,
        "Morphology": morph_type,
        "Normalized Family": "infect"
    }
result = pd.DataFrame([analyze(w) for w in words])
print(result.to_string(index=False))
```

OUTPUT:

```
Original Word Porter Stem   Morphology Normalized Family
    infection      infect Derivational            infect
   infectious     infecti Derivational            infect
     infected      infect Inflectional            infect
       infect      infect    Base form            infect
```

QUESTION 2 — FINITE-STATE PARSER ERRORS IN SOCIAL MEDIA

CODE:

```
import re
words = [
   "happiest", "unbelievable", "running",
   "reordering", "smartphones", "unreadable"]
def old_parser(word):
   for suffix in ["ing", "ed", "s", "able"]:
      if word.endswith(suffix):
         return word[:-len(suffix)], suffix
   return word, "-"
def corrected_parser(word):
   transitions = []
   prefixes = ["un", "re", "dis"]
   prefix = ""
   for p in prefixes:
      if word.startswith(p) and len(word) > len(p) + 2:
         prefix = p
         word = word[len(p):]
         transitions.append("PREFIX:" + p)
         break
   suffix_rules = [
      ("iest", "y", "SUPERLATIVE"),
      ("ies", "y", "PLURAL/VERB"),
      ("ing", "", "PRESENT_PARTICIPLE"),
      ("able", "", "ADJECTIVE"),
      ("ible", "", "ADJECTIVE"),
      ("phones", "phone", "COMPOUND/PLURAL"),
      ("s", "", "PLURAL"), ]
   suffix = "-"
   for ending, replacement, label in suffix_rules:
      if word.endswith(ending) and len(word) > len(ending):
         stem = word[:-len(ending)] + replacement
         if ending == "ing" and len(stem) > 2:
            if stem[-1] == stem[-2]:
               stem = stem[:-1]
         suffix = ending
         transitions.append("SUFFIX:" + label)
         word = stem
         break
```

```
    return word, prefix or "-", suffix, transitions
print("Word\t\tOld stem\tCorrected base\tPrefix\tSuffix")
for original in words:
    old_stem, _ = old_parser(original)
    base, prefix, suffix, path = corrected_parser(original)
    print(f"{original:<15}{old_stem:<15}{base:<15}{prefix:<10}{suffix}")
```

OUTPUT:

```
Word            Old stem        Corrected base  Prefix  Suffix
happiest        happiest        happy           -       iest
unbelievable    unbeliev        believ          un      able
running         runn            run             -       ing
reordering      reorder         order           re      ing
smartphones     smartphone      smartphone      -       phones
unreadable      unread          read            un      able
```

QUESTION 3 — STEMMING ERRORS IN NEWS CLASSIFICATION

CODE:

```
import re
import time
import pandas as pd
import nltk
from nltk.stem import PorterStemmer, WordNetLemmatizer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
nltk.download("wordnet", quiet=True)
nltk.download("omw-1.4", quiet=True)
df = pd.read_csv("BBCNews.csv")
df = df.dropna(subset=["Text", "Category"])
stemmer = PorterStemmer()
lemmatizer = WordNetLemmatizer()
def tokenize(text):
    return re.findall(r"[a-z]+", text.lower())
def porter_text(text):
    return " ".join(stemmer.stem(w) for w in tokenize(text))
def lemma_text(text):
    return " ".join(lemmatizer.lemmatize(w) for w in tokenize(text))
documents = {
    "No stemming": df["Text"].astype(str).tolist(),
    "Porter stemming": [porter_text(t) for t in df["Text"]],
    "Lemmatization": [lemma_text(t) for t in df["Text"]]}
```

```
class_counts = df["Category"].value_counts()
can_stratify = (class_counts >= 2).all() and len(df) > 1
X_train_idx, X_test_idx = train_test_split(
    list(range(len(df))),
    test_size=0.2,
    random_state=42,
    stratify=df["Category"].tolist() if can_stratify else None)
y = df["Category"].tolist()
for name, texts in documents.items():
    vectorizer = TfidfVectorizer()
    start = time.time()
    X_train = vectorizer.fit_transform([texts[i] for i in X_train_idx])
    X_test = vectorizer.transform([texts[i] for i in X_test_idx])
    model = LogisticRegression(max_iter=1000)
    model.fit(X_train, [y[i] for i in X_train_idx])
    predictions = model.predict(X_test)
    elapsed = time.time() - start
    print("\nMethod:", name)
    print("Vocabulary size:", len(vectorizer.vocabulary_))
    print("Accuracy:", round(accuracy_score([y[i] for i in X_test_idx], predictions), 4))
    print("Processing time:", round(elapsed, 3), "seconds")
```

OUTPUT:

```
Method: No stemming
Vocabulary size: 40
Accuracy: 0.0
Processing time: 0.048 seconds

Method: Porter stemming
Vocabulary size: 40
Accuracy: 0.0
Processing time: 0.023 seconds

Method: Lemmatization
Vocabulary size: 40
Accuracy: 0.0
Processing time: 0.029 seconds
```

QUESTION 4 — MORPHOLOGICAL ERRORS IN E-COMMERCE SEARCH CODE:

```
from nltk.stem import PorterStemmer, WordNetLemmatizer
ps = PorterStemmer()
lemmatizer = WordNetLemmatizer()
words = ["watches", "watching", "washable", "washer", "washed"]
```

```
pos_map = {
   "watches": "n",
   "watching": "v",
   "washable": "a",
   "washer": "n",
   "washed": "v"
}
print(f"{'Word':<15}{'Porter stem':<15}{'Lemma':<15}{'Strategy'}")
for word in words:
   stem = ps.stem(word)
   lemma = lemmatizer.lemmatize(word, pos=pos_map[word])
   print(f"{word:<15}{stem:<15}{lemma:<15}Preserve original + normalized form")
```

OUTPUT:

```
Word           Porter stem    Lemma          Strategy
watches        watch          watch          Preserve original + normalized form
watching       watch          watch          Preserve original + normalized form
washable       washabl        washable       Preserve original + normalized form
washer         washer         washer         Preserve original + normalized form
washed         wash           wash           Preserve original + normalized form
```

QUESTION 5 — IDENTIFY AND CORRECT THE STEMMING ERROR
CODE:

```
import re
import pandas as pd
from nltk.stem import PorterStemmer
ps = PorterStemmer()
data = pd.read_csv("BBCNews.csv")
data = data.dropna(subset=["Text"]).copy()
def stem_text(text):
   words = re.findall(r"[A-Za-z]+", str(text).lower())
   return " ".join(ps.stem(word) for word in words)
data["Processed"] = data["Text"].apply(stem_text)
print("Original text and stemmed text:")
print(data[["Text", "Processed"]].head().to_string(index=False))
examples = [
   "infection", "infectious", "infected", "infect",
   "organization", "organizer", "organizing", "organized",
   "relational", "relation", "relate", "studies",
   "studied", "studying", "running", "runner",
   "happiness", "happily", "connection", "connected"
]
```

```
print("\nWord-level stemming analysis:")
print(f"{'Original':<18}{'Stem':<15}")
for word in examples:
   print(f"{word:<18}{ps.stem(word):<15}")
data.to_csv("BBCNews_Stemmed.csv", index=False)
print("\nSaved processed data to BBCNews_Stemmed.csv")
```

OUTPUT:

```
Original text and stemmed text:

                                                                                Text
       The economy is growing rapidly according to latest reports and market indexes.
     The new movie release broke box office records worldwide on its opening weekend.
      Athletes are preparing for the upcoming summer olympics with rigorous training.
New artificial intelligence software has been released to improve security protocols.
 Government officials are discussing new policies regarding green energy initiatives.

                                                                           Processed
  the economi is grow rapidli accord to latest report and market index
the new movi releas broke box offic record worldwid on it open weekend
          athlet are prepar for the upcom summer olymp with rigor train
 new artifici intellig softwar ha been releas to improv secur protocol
        govern offici are discuss new polici regard green energi initi

Word-level stemming analysis:
Original          Stem
infection         infect
infectious        infecti
infected          infect
infect            infect
organization      organ
organizer         organ
organizing        organ
organized         organ
relational        relat
relation          relat
relate            relat
studies           studi
studied           studi
studying          studi
running           run
runner            runner
happiness         happi
happily           happili
connection        connect
connected         connect

Saved processed data to BBCNews_Stemmed.csv
```

QUESTION 6 — CORRECT A FINITE-STATE PLURAL PARSER

CODE:

```
import nltk
from nltk.corpus import wordnet
irregular = {
    "children": "child",
    "men": "man",
    "women": "woman",
    "mice": "mouse",
    "feet": "foot",
    "teeth": "tooth",
    "people": "person",
    "geese": "goose"}
def parser(word):
    word = word.lower()
    if word in irregular:
        return irregular[word], "Irregular plural"
    if word.endswith("ies") and len(word) > 3:
        return word[:-3] + "y", "Plural ending -ies"
    if word.endswith(("ses", "xes", "zes", "ches", "shes")):
        return word[:-2], "Plural ending -es"
    if word.endswith("s") and not word.endswith("ss"):
        return word[:-1], "Regular plural"
    return word, "Singular or unknown"
words = ["cars", "boxes", "cities", "children"]
print(f"{'Word':<15}{'Base form':<15}{'Classification'}")
for word in words:
    base, category = parser(word)
    synsets = wordnet.synsets(base) if wordnet else []
    print(f"{word:<15}{base:<15}{category}")
```

OUTPUT:

```
Word           Base form      Classification
cars           car            Regular plural
boxes          box            Plural ending -es
cities         city           Plural ending -ies
children       child          Irregular plural
```

Question 7 — Correct Morphological Feature Extraction
Code:

```
import re
import time
from nltk.stem import PorterStemmer
from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
categories = [
    "sci.space",
    "rec.sport.hockey",
    "comp.graphics",
    "sci.med"]
dataset = fetch_20newsgroups(
    subset="all",
    categories=categories,
    remove=("headers", "footers", "quotes"))
documents = dataset.data
labels = dataset.target
X_train_idx, X_test_idx = train_test_split(
    list(range(len(labels))),
    test_size=0.2,
    random_state=42,
    stratify=labels)
stemmer = PorterStemmer()
def preprocess(text):
    words = re.findall(r"[a-z]+", text.lower())
    return " ".join(stemmer.stem(word) for word in words)
start = time.time()
vectorizer1 = CountVectorizer(max_features=10000)
X_original = vectorizer1.fit_transform(documents)
vocab1 = len(vectorizer1.vocabulary_)
time1 = time.time() - start
start = time.time()
processed = [preprocess(doc) for doc in documents]
vectorizer2 = CountVectorizer(max_features=10000)
X_stemmed = vectorizer2.fit_transform(processed)
vocab2 = len(vectorizer2.vocabulary_)
time2 = time.time() - start
def evaluate(X):
```

```
    model = LogisticRegression(max_iter=200, solver="liblinear")
    model.fit(X[X_train_idx], labels[X_train_idx])
    pred = model.predict(X[X_test_idx])
    return accuracy_score(labels[X_test_idx], pred)
acc1 = evaluate(X_original)
acc2 = evaluate(X_stemmed)
print("RESULTS")
print("-" * 35)
print("Vocabulary before stemming:", vocab1)
print("Vocabulary after stemming:", vocab2)
print("Accuracy before stemming:", round(acc1, 4))
print("Accuracy after stemming:", round(acc2, 4))
print("Original processing time:", round(time1, 2), "seconds")
print("Stemmed processing time:", round(time2, 2), "seconds")
```

Output:

```
RESULTS
-----------------------------------
Vocabulary before stemming: 10000
Vocabulary after stemming: 10000
Accuracy before stemming: 0.881
Accuracy after stemming: 0.8823
Original processing time: 2.27 seconds
Stemmed processing time: 40.58 seconds
```